Introduction to Artificial Intelligence

Tutorial Sheet 2 (Model answers)

Exercise 1

Let us examine the rationality of various vacuum-cleaner agent functions.

a. Show that the simple vacuum-cleaner agent function is indeed rational under the assumptions:

- The performance measure awards one point for each clean square at each time step, over a "lifetime" of 1000-time steps.
- The "geography" of the environment is known a priori (Figure 2.2 and 2.3 below) but the dirt distribution and the initial location of the agent are not. Clean squares stay clean and sucking cleans the current square. The Left and Right actions move the agent left and right except when this would take the agent outside the environment, in which case the agent remains where it is.
- The only available actions are *Left*, *Right*, and *Suck*.
- The agent correctly perceives its location and whether that location contains dirt.
- The goal of the agent is to have the two squares clean.

b. Describe a rational agent function for the case in which each movement costs one point. Does the corresponding agent program require an internal state?

c. Discuss possible agent designs for the cases in which clean squares can become dirty and the geography of the environment is unknown. Does it make sense for the agent to learn from its experience in these cases? If so, what should it learn? If not, why not?

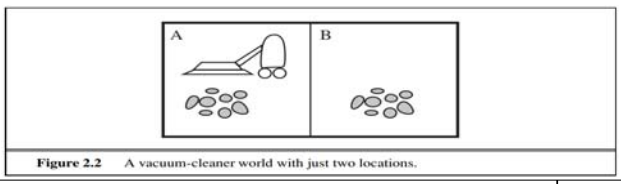


Figure 2.2 A vacuum-cleaner world with just two locations.

Percept sequence	Action
$[A, Clean]$	$Right$
$[A, Dirty]$	$Suck$
$[B, Clean]$	$Left$
$[B, Dirty]$	$Suck$
$[A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Dirty]$	$Suck$
⋮	⋮
$[A, Clean], [A, Clean], [A, Clean]$	$Right$
$[A, Clean], [A, Clean], [A, Dirty]$	$Suck$
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

a. Show that the simple vacuum-cleaner agent function is indeed rational.

First, what is a rational agent function ➔ an agent function is considered rational if it selects actions that are expected to achieve its goals based on the available information (perform the task that it has been designed to achieve) while maximizing some utility function.

It suffices to show that for all possible actual environments (i.e., all dirt distributions and initial locations), **this agent cleans the squares at least as fast as any other agent**.

All cases:

No dirt in both squares → optimal after zero steps

One clear, one dirty, agent is in the dirty square → optimal after one step

One clear, one dirty, agent is in the clean square → optimal after two steps

Two dirty squares → optimal after three steps

➔No type of agent can do better!!

b. Describe a rational agent function for the case in which each movement costs one point. Does the corresponding agent program require an internal state?

The agent in (a) keeps moving backwards and forwards even after the world is clean. It is better to do **NoOp** once the world is clean (the chapter says this). Now, since the agent's percept doesn't say whether the other square is clean**, it would seem that the agent must have some memory to say whether the other square has already been cleaned**.

For the case in which each movement costs one point, **the agent should stop after checking both squares A and B and removing any dirt found in order to reduce needless movements and loss of points (choosing NoOp)**.

➔The corresponding agent program would require an internal state in this case, as it would need to remember checking both squares before it stops performing its task. If it finds dirt at square A, cleans it, then moves to square B and finds no dirt, and then if it does not remember whether or not it checked square A, it will check A again, and then repeat with square B and continue making unnecessary movements, thus losing points which would irrational with respect to the utility function.

➔For the case in which clean squares can become dirty, the agent could work at some fixed time interval. It could check and clean both squares A and B, then stop for a fixed amount of time, then start up and repeat the process in order to keep the squares clean while minimizing needless moves and performance measurement point loss.

c. Discuss possible agent designs for the cases in which clean squares can become dirty and the geography of the environment is unknown. Does it make sense for the agent to learn from its experience in these cases? If so, what should it learn? If not, why not?

➔If the geography of the environment is unknown, the agent would need to explore the environment. **It could learn the geography of the environment by exploring it (using exploitation strategies to be seen next chapter)**.

➔Learning from experience could be beneficial in these cases, as the agent can gather information about the environment dynamics and adjust its behavior accordingly. The agent could learn various things, such as the likelihood of squares becoming dirty, the effectiveness of different cleaning strategies in different environments, and optimal navigation paths. Reinforcement learning algorithms could be suitable for this task, allowing the agent to learn from rewards (clean squares) and adjust its behavior over time to maximize long-term performance.

Exercise 2

For each of the following assertions, say whether it is true or false and support your answer with examples or counterexamples where appropriate.

a. An agent that senses only partial information about the state cannot be perfectly rational.

b. There exist task environments in which no pure reflex agent can behave rationally.

c. There exists a task environment in which every agent is rational.

d. The input to an agent program is the same as the input to the agent function.

e. Every agent function is implementable by some program/machine combination.

f. Suppose an agent selects its action uniformly at random from the set of possible actions. There exists a deterministic task environment in which this agent is rational.

g. It is possible for a given agent to be perfectly rational in two distinct task environments, i.e. where each environment has its own set of characteristics, constraints, and objectives.

h. Every agent is rational in an unobservable environment.

i. A perfectly rational poker-playing agent never loses.

j. For a simple reflex agent in a partially observable environment, a randomized policy can outperform any deterministic policy.

k. There is a model-based reflex agent that can remember all of its percepts.

l. Suppose agent A1 is rational and agent A2 is irrational. There exists a task environment where A2's actual score will be greater than A1's actual score.

a. An agent that senses only partial information about the state cannot be perfectly rational.

False. **Perfect rationality refers to the ability to make good decisions, i.e. decisions that maximize a utility function, using the received sensor information (and available knowledge).** For example, a weather forecasting agent that only has access to limited data can still make rational predictions about the weather based on that information.

Or a medical diagnosis system that analyzes a patient's symptoms but lacks access to their full medical history. Despite this partial information, the system can still make rational diagnoses by applying probabilistic reasoning and pattern recognition techniques.

b. There exist task environments in which no pure reflex agent can behave rationally.

True. A pure reflex agent ignores previous percepts, so it cannot obtain an optimal state estimate.

Example: A chess-playing agent based solely on reflex actions cannot adapt to changing strategies or hidden information, which would lead to irrational moves.

c. There exists a task environment in which every agent is rational.
True. For example, a very simple environment with a single state, such that all actions have the same reward: it doesn't matter which action is taken. More generally, any environment that is reward-invariant under permutation of the actions will satisfy this property (exercise 1, question 1 is also the case).

d. The input to an agent program is the same as the input to the agent function.
False.

- The agent function, in a conceptual sense, takes the entire percept sequence up to the current time step as its input. This means it has full knowledge of the **history of the environment** (i.e., the sequence of inputs or percepts received by the agent over time) and uses this entire history to determine the next action.
- The agent program, in its concrete implementation, takes only the current percept as input. It then uses its internal representation or implementation of the agent function to predict the next action based on the current percept.

Why This Difference Exists:

The distinction comes down to the practical **limitations of real-world implementations**. In theory, an agent function can use the entire percept sequence because it has unlimited access to all past data. In practice, though, agent programs often operate under constraints:

- **Memory**: Storing all percepts may be infeasible or inefficient.
- **Computational Limitations**: Processing a long history of percepts can be too slow.
- **Real-time Decision Making**: In many real-world applications, the agent must decide its next action with limited or real-time information (just the current percept).

As a result, the **agent program** needs to approximate or "simulate" the behavior of the ideal agent function, using only the current percept and any additional internal state.

e. Every agent function is implementable by some program/machine combination.
False. **Limitations of computation:** There are certain problems that are fundamentally **uncomputable** due to limitations of computation itself. These problems, no matter how cleverly designed a program or machine might be, cannot be solved in a finite amount of time or resources.

- **Halting problem:** The classic example is the **halting problem**, which asks whether a given Turing machine will eventually halt or run forever on a given input tape. This problem is proven to be incomputable, meaning there is no program that can definitively answer this question for all possible Turing machines and input tapes.
- **Intractability:** Another limitation comes from problems that are **intractable**, meaning they become exponentially harder to solve as the problem size increases. Even with the most powerful computers, solving these problems for large instances becomes impractical. Imagine an agent function that requires checking all possible combinations of actions in a complex environment in constant time - it's simply not possible due to the inherent complexity of the problem.

Examples of unimplementable agent functions:

- An agent function that tells you whether a random string represents a valid program (which is also related to the halting problem).
- An agent function that determines the optimal chess move in constant time, regardless of the board configuration (chess is known to be computationally intractable).
- An agent function that predicts the future with perfect accuracy (predicting the future perfectly is impossible due to inherent randomness in the world).

f. Suppose an agent selects its action uniformly at random from the set of possible actions. There exists a deterministic task environment in which this agent is rational.
True. if it doesn't matter which action you take, selecting randomly is rational.
Example: Imagine a very simple environment with only two states and two actions. In each state, both actions lead to the same reward, say 1 point. In this case, choosing an action randomly is just as good as choosing any specific action, and the agent will achieve the optimal score regardless of its choice.

Let's say we have an environment where the agent has to make a decision in two different rooms (states). Each room contains the same type of object (let's call them "tokens"), and the agent can either pick up a token (Action A) or leave it (Action B). However, in this simple environment, it doesn't matter whether the agent picks up a token or leaves it because both actions give the same reward.

Details of the environment:

1. **State 1 (Room 1)**: The agent is in Room 1.
   - Action A: Pick up the token → Reward = 1 point.
   - Action B: Leave the token → Reward = 1 point.

   (Both actions give the same reward, 1 point.)

2. **State 2 (Room 2)**: The agent is in Room 2.
   - Action A: Pick up the token → Reward = 1 point.
   - Action B: Leave the token → Reward = 1 point.

   (Again, both actions give the same reward, 1 point.)

Agent Behavior:
The agent can choose either Action A or Action B randomly in each state because both actions result in the same reward.

- In **State 1**, it doesn't matter whether the agent picks up the token or leaves it, as both actions will result in 1 point.
- In **State 2**, the same applies: both Action A and Action B lead to the same reward of 1 point.

Outcome:

- The agent will always end up with 1 point for each state, regardless of which action it takes.
- If the agent chooses randomly, it will still receive the optimal reward (1 point) in both states.

Why This Is Rational:

- A rational agent is one that maximizes its expected reward. In this case, since both actions in both states lead to the same reward, it doesn't matter which action the agent chooses.
- **Choosing randomly** is still rational because the agent will achieve the same outcome (1 point) regardless of its decision.

In this environment, **random action selection** is **as good as any deterministic strategy** (e.g., always choosing Action A or always choosing Action B) since the agent cannot improve its outcome.

g. It is possible for a given agent to be perfectly rational in two distinct task environments, i.e. where each environment has its own set of characteristics, constraints, and objectives.
True. If two task environments have compatible performance measures and dynamics, the same agent can be perfectly rational in both environments.
Consider a scenario where a robot vacuum cleaner operates in two distinct rooms, each with its unique environment, including layouts of obstacles such as furniture and walls. In both rooms, the robot's objective remains the same: to clean as much of the room as possible within a specified time frame. Despite the differences in the environment layouts between the two rooms, the robot can employ an exploration strategy that enables it to effectively clean both rooms. This strategy allows the robot to adapt its cleaning behavior based on the specific characteristics of each room, maximizing its cleaning performance regardless of the variations in the environments.

h. Every agent is rational in an unobservable environment.
False. Some actions are stupid (ex. Doing nothing) and the agent may know this if it has a model of the environment—even if one cannot perceive the environment state.

i. A perfectly rational poker-playing agent never loses.
False. Unless it draws the perfect hand, the agent can always lose if an opponent has better cards. This can happen for game after game. The correct statement is that a perfectly rational poker-playing agent's expected winnings are nonnegative.

j. For a simple reflex agent in a partially observable environment, a randomized policy can outperform any deterministic policy.
True. a simple reflex agent may not have enough information to make optimal decisions deterministically. Introducing randomness through a randomized policy can allow the agent to explore different actions and potentially discover better strategies than a deterministic policy constrained by limited perceptual information.

Example: **Game of Rock-Paper-Scissors**: In this game, if a player (the agent) always chooses the same move (deterministic policy), an opponent can easily predict and defeat the agent. However, if the agent uses a randomized policy, choosing rock, paper, or scissors at random, the opponent cannot predict the agent's move, and the agent has a better chance of winning over time.

k. There is a model-based reflex agent that can remember all of its percepts.
True. A model-based agent updates an internal state representation—its "memory"— for each new percept. Memorizing the percepts is possible (assuming unbounded memory). This may not be the most perspicuous representation of the current state of the world, but any other such representation of the current state could be computed from it on demand, for example by running any other model-based reflex agent's state estimation algorithm on the memorized percept sequence.

l. Suppose agent A1 is rational and agent A2 is irrational. There exists a task environment where A2's actual score will be greater than A1's actual score.
True. Rational decisions are defined by expected outcomes, not actual outcomes. A1 might just be unlucky.
Example: Stock market prediction.
A1 might invest in businesses based on the predictions which are usually very scientific and reliable. But A2 might be lucky and get a better Return on Investment by investing on businesses that may not appear to be the most promising if the stock market predictions are considered.

Exercise 3
For each of the following activities, give a PEAS (Performance, Environment, Actuators, Sensors) description of the task environment and characterize it in terms of the properties (Section 2.3.2 of the textbook): **Environment observability**: fully vs partially observable; **Environmental predictability**: Deterministic vs. stochastic; **Task structure**: Episodic vs. sequential; **Environmental stability**: Static vs. dynamic; **Action granularity**: Discrete vs. continuous; **Number of agents**: single vs multi-agent.

1. **Playing soccer:**
   - Partially observable: Players cannot see the entire field or know the exact positions and intentions of all players and the ball at all times.
   - Stochastic: The outcomes of actions (e.g., kicking the ball, passing to a teammate) are not entirely predictable due to factors like wind, surface conditions, and the actions of other players.
   - Sequential: Soccer consists of a sequence of actions and decisions, where each decision can affect future actions and outcomes.
   - Dynamic: The environment changes continuously as players and the ball move around the field.
   - Continuous: Player actions like running, kicking, and passing involve continuous motion and force.
   - Multi-agent: Soccer involves multiple players on each team, who can be considered agents interacting with each other and the environment. The agents from each team have a common goal and utility function.

2. **Exploring the subsurface oceans of Titan:**
   investigating the vast liquid oceans believed to exist beneath the icy surface of Saturn's moon which is called Titan (*Titan* is the largest *moon* of ***Saturn*** the sixth planet from the Sun*).*
   - Partially observable: The exploration vehicle can only perceive a limited area around it, and there may be unseen aspects of the environment.
   - Stochastic: The effects of actions (e.g., moving, collecting samples) may be influenced by factors like currents, terrain, and other unpredictable environmental factors.
   - Sequential: The exploration involves a sequence of actions and decisions, where each action can affect future observations and options.
   - Dynamic: The environment may change due to factors like currents, weather, or geological activity.
   - Continuous: The exploration vehicle's movement and sensor readings involve continuous values.
   - Single agent: Unless there are alien life forms that can be modeled as agents, the exploration vehicle is considered a single agent operating in the environment.

3. **Shopping for used AI books on the Internet:**
   - Partially observable: The buyer may not have complete information about all available books, their conditions, or shipping times.
   - Deterministic: The outcomes of actions (e.g., purchasing a book) are generally predictable and not influenced by random factors.
   - Sequential: Book shopping involves a sequence of actions and decisions, where each decision can affect future options and outcomes. E.g. purchasing a book, if it is the last copy available of that title, will make it not available until new copies are brought into the stock.
   - Dynamic: The store on the internet is being used by thousands (even more) at each instant (just like the taxi-driving agent on a motorway). As one agent is thinking about purchasing a book, others may purchase it, and the title may no longer be available when this agent decides to order it.
   - Discrete: Actions like browsing, selecting, and purchasing books are discrete events.
   - Multi-Agent: See the above explanation about the Dynamic nature of the environment.

4. **Playing a tennis match:**
   - Fully observable: Both players can see the entire court, the ball, and each other's actions.
   - Stochastic: The outcomes of actions (e.g., serving, hitting the ball) are influenced by factors like wind, surface conditions, and the opponent's actions.
   -
   - Sequential: The actions are not points; they are about moving on the court, hitting the ball (possibly in various ways), etc. Each of these actions will have a future impact on winning a point or not, hence eventually on sets, and the game. Even a move to a position may favor (or not) the action of hitting the ball right after the move. (Again, as for a taxi-driving agent, it may change lanes, accelerate, decelerate, etc., the end goal being to reach some destination; in this case winning a point.)
   - Dynamic: The environment (ball and player positions) changes continuously during action being taken.
   - Continuous: Player actions like swinging the racket, running, and hitting the ball involve continuous motion and force. (Note that this answer which mentions the actions confirms the point about the Sequential nature of the environment.)
   - Multi-agent: Tennis involves two players who can be considered agents interacting with each other and the environment. Each player has a goal and a utility function.

5. **Practicing tennis against a wall:**
   - Fully observable: The player (agent) can see the entire practice area, the ball, and its own actions.
   - Stochastic: The outcomes of actions (e.g., hitting the ball against the wall) may be influenced by factors like wind, surface conditions, and the way the ball bounces off the wall.
   - Sequential: I do not agree here either. The same argument as in the previous agent o laying tennis makes the environment Sequential here again. (The argument is in terms of actions.)
   - Dynamic: The environment (ball position) changes continuously during each action being taken.
   - Continuous: Player actions like swinging the racket and hitting the ball involve continuous motion and force.
   - Single agent: The player is practicing alone, so there is only a single agent interacting with the environment.

6. **Performing a high jump:**
   - Fully observable: The athlete can see the entire jumping area, the bar, and its own actions.
   - Stochastic: The outcome of the jump may be influenced by factors like wind, surface conditions, and the athlete's execution of the jump.
   - Sequential: The high jump involves a sequence of actions (approach, take-off, bar clearance, landing) that build upon each other. (Comment: This is correct; see the parallel with playing tennis where the (sub)goals are winning points while here it is about jumping over the horizontal bar.)
   - Static: The environment (jumping area, bar height) does not change during a single jump attempt.
   - Continuous: The athlete's actions like running, jumping, and clearing the bar involve continuous motion and force.
   - Single agent: The high jumper is a single agent interacting with the environment.

7. **Knitting a sweater:**
   - Fully observable: The knitter can see the entire work area, the yarn, the needles, and their own actions.
   - Deterministic: The outcomes of actions (e.g., knitting stitches) are predictable and not influenced by random factors.
   - Sequential: Knitting a sweater involves a sequence of actions and decisions, where each step builds upon the previous ones.
   - Static: The environment (knitting materials, pattern) does not change during the knitting process.
   - Continuous: The knitter's actions like manipulating the yarn and needles involve continuous motion and force.
   - Single agent: The knitter is a single agent interacting with the environment.

8. **Bidding on an item at an auction:**
   - Fully observable: The bidders can see the current bid, the item, and the actions of other bidders.
   - Strategic: The outcomes of actions (e.g., placing a bid) are influenced by the decisions and actions of other bidders, who can be considered strategic agents.
   - Sequential: The auction involves a sequence of bids and decisions, where each bid can affect future actions and outcomes.
   - Static: The environment (item, auction rules) does not change during the auction process.
   - Discrete: Actions like placing bids are discrete events.
   - Multi-agent: The auction involves multiple bidders who can be considered agents interacting with each other and the environment.

Exercise 4
Consider a modified version of the vacuum environment in Exercise 1, in which the agent is penalized one point for each movement.

a. Can a simple reflex agent be perfectly rational for this environment? Explain.

No, For the case in which each movement costs one point, **the agent should stop after checking both squares A and B and removing any dirt found in order to reduce needless movements and loss of points (choosing NoOp)**.
In general, the problem with reflex agents is that they have to do the same thing in situations that look the same, even when the situations are actually quite different.

b. What about a reflex agent with state? Design such an agent.
Yes, a reflex agent with state can be perfectly rational. It simply needs to remember whether it has visited the other square. If so, it can do nothing after cleaning the current square. If not, it then goes to the other square and cleans it if needed.

c. How do your answers to A and B change if the agent's percepts give it the clean/dirty status of every square in the environment?
- Can a simple reflex agent be perfectly rational for this environment? Explain.

Yes, The agent's decision-making process is based solely on the current percept (the agent's location and the state of the all the squares in the environment). With perfect information, the agent can construct a table that maps each possible percept (agent's location and clean/dirty status of the all the squares) to the appropriate action (Left, Right, etc., or Suck).
➔ For a small environment (in this case, the environment is small with only two squares), the agent can construct a table with all possible percepts and corresponding actions.

Percept			Action
Current position	**State of A**	**State of B**	**Action**
A	Clean	Clean	NoOp
A	Clean	Dirty	Right
A	Dirty	Clean	Suck
A	Dirty	Dirty	Suck
B	Clean	Clean	NoOp
B	Clean	Dirty	Suck
B	Dirty	Clean	Left
B	Dirty	Dirty	Suck

as the environment grows larger, constructing a table for every possible percept becomes infeasible due to the exponential growth in the number of possible percepts. In such cases, the agent can employ optimal search algorithms (chapter 3) to determine the best action based on the current percept and the knowledge of the entire environment.

- What about a reflex agent with state (i.e. a reflex agent that takes into account both the current percept and the history of past actions, so it keeps track of its internal state, which is updated as it interacts with the environment)?

In larger environments, constructing a table to map every possible percept-action pair can become impractical due to the sheer number of combinations. Instead of relying solely on reactive responses to percepts, the agent with state can use its history to anticipate future consequences of actions, leading to more efficient decision-making.

Exercise 5
The vacuum environments in the preceding exercises have all been deterministic. Discuss possible agent programs for each of the following stochastic versions:

a. Murphy's law: twenty-five percent of the time, the Suck action fails to clean the floor if it is dirty and deposits dirt onto the floor if the floor is clean. How is your agent program affected if the dirt sensor gives the wrong answer 10% of the time?

- **Reflex Agent:**
  - The reflex agent's behavior remains largely unaffected by the additional uncertainties introduced by Murphy's law and the unreliable dirt sensor. It will continue to execute the Suck action as long as the current location remains dirty, regardless of potential failures or incorrect sensor readings.
- **Agent with Sequential Plan:**
  - In contrast, an agent that constructs a sequential plan needs to adapt its strategy to cope with Murphy's law and the unreliable sensor.
  - Every Suck action in the sequential plan must be replaced by "Suck until clean" to ensure that the floor is adequately cleaned despite the uncertainties.
  - If the dirt sensor can be wrong on each step, the agent may adopt a strategy of waiting for a few steps to gather more reliable information before deciding whether to Suck or move on to a new square. However, there's a trade-off involved: waiting too long may result in penalties due to dirt accumulation, while acting immediately risks dirtying a clean square or ignoring a dirty square if the sensor is wrong.
  - A rational agent must continue touring and checking the squares in case it missed one on a previous tour due to bad sensor readings. Determining the optimal waiting time at each square with each new tour is not straightforward and may require experimentation and mathematical analysis.
  - This problem is a partially observable Markov decision process (POMDP), which is generally challenging to solve. However, some special cases may yield to careful analysis. (For a POMDP, methods are used which aim to handle the uncertainty due to partial observability and make decisions that maximize the expected long-term reward, representing the states as belief states, i.e. a probability distribution over possible states is maintained to represent the agent's knowledge about the environment.)

b. Small children: At each time step, each clean square has a 10% chance of becoming dirty. Can you come up with a rational agent design for this case?

- In this stochastic environment, where each clean square has a 10% chance of becoming dirty at each time step, the agent faces a different set of challenges.
- The rational strategy for the agent is to keep touring the squares indefinitely. Since the probability that a square is dirty increases monotonically with the time since it was last cleaned, the optimal strategy is to repeatedly execute the shortest possible tour of all squares.
- However, complications arise because the shortest tour may visit some squares twice, depending on the geography of the environment.
- Despite these complications, the agent's goal remains clear: to continuously tour the environment and clean dirty squares efficiently.
- Similar to the Murphy's law scenario, this problem is also a partially observable Markov decision process (POMDP), which presents challenges in finding optimal solutions but can potentially be addressed through careful analysis.